
Práctica 2: Espacio de pantalla y z-buffer

Fecha de entrega: 3 de febrero de 2026, 15:40

Objetivo: Espacio de pantalla y z-buffer

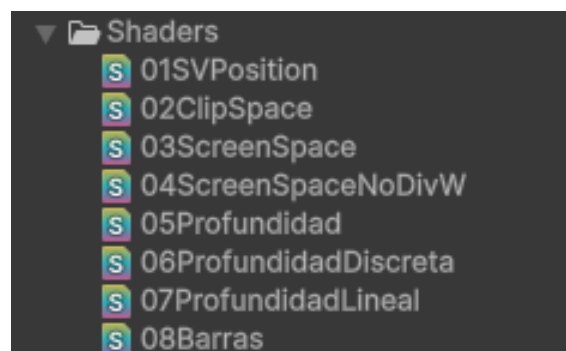
1. Instrucciones

[Iguales a las de la práctica anterior]

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo shaders y materiales que se aplican sobre un cubo.

La entrega debe incluir únicamente los shaders individuales con los nombres *de ficheros* concretos indicados en el texto y una captura de la pestaña *Game* del editor de Unity en la que se vea el cubo con el material aplicado. Es importante que los nombres de los ficheros entregados coincidan con los pedidos. En el enunciado también se especifica qué nombre deben tener las variables HLSL de los parámetros del material.

Si vas creando los shaders en la carpeta `Shaders` del proyecto, al final de la práctica deberías ver algo así:



En la entrega habrá un fichero con extensión `.shader` por cada uno de los archivos que aparecen en la imagen, así como una captura del resultado de aplicar ese material con

extensión `.png` o `.jpg`. Por ejemplo, se espera un fichero `01SVPosition.shader` y un `01SVPosition.png` (o `.jpg`).

Para el desarrollo se recomienda crear la escena base con el cubo y duplicarla de forma que en cada escena el cubo utilice un material y shader distinto.

2. Punto de partida

[Igual al de la práctica anterior]

Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline* con una escena con:

- El contenido por defecto de la plantilla de Unity, con skybox, cámara y luz direccional.
- La cámara con la localización (L), escalado (S) y rotación (R) siguientes: L: $(-3, 1, -3)$, S: $(1, 1, 1)$, R: $(10, 45, 0)$.
- Un cubo en L: $(0, 0, 0)$, S: $(1, 1, 1)$, R: $(0, 0, 0)$.

3. Dibujando el **SV_POSITION**

Crea un *Unlit shader* simple llamado `01SVPosition` que nos sirva para “dibujar” el *clip space* que recibe el PS en el `float4` de semántica `SV_POSITION`.

El rango del *clip space* puede variar dependiendo de la plataforma pero lo normal es que la *X* e *Y* estén entre -1 y 1. Haz que el PS devuelva el color $(x, y, 0, 1)$. Recuerda que el *clip space* está en *coordenadas homogéneas*, por lo que hay que dividir entre la componente *w*:

```
return float4(abs(i.pos.xy / i.pos.w), 0.0f, 1.0f);
```

¿Qué resultado esperas?

4. Dibujando el clip space

Duplicando el shader anterior, crea otro `02ClipSpace` para hacer lo mismo pero haciendo que el VS pase el *clip space* dos veces, uno con semántica `SV_POSITION` como antes y otro con semántica `TEXCOORD0`¹. El VS debe dar a las dos *el mismo valor* (`o.pos = o.pos2 = TransformObject...`).

Haz que el valor devuelto por el PS se calcule igual que antes pero utilizando el nuevo campo. ¿Qué resultado esperas? ¿Qué conclusiones sacas de lo que ves? Para mayor comodidad, prueba a ver el cubo en distintos sitios de la ventana en la pestaña de la escena.

Entrega una captura *de la pestaña de la escena* habiendo colocado la cámara cerca del cubo para que éste lo único que se vea.

¹Aunque la semántica `TEXCOORD0` suele asociarse a coordenadas de textura de tipo `float2`, nada nos impide que el tipo sea `float4`.

5. Dibujando el screen space

Duplicando el shader, crea otro `03ScreenSpace` para hacer lo mismo pero utilizando el espacio de pantalla (*screen space*, SS). En Unity la función que convierte de CS a SS es `ComputeScreenPos` que recibe y devuelve un `float4` (coordenadas homogéneas). Fíjate que ya no es necesario el `abs` pues en SS tanto la x como la y están entre 0 y 1.

Haz dos versiones (no hagas dos shaders; deja la primera versión comentada y entrega solo la segunda):

- Una primera versión en la que es el PS el que calcula, para la posición en CS que recibe, la posición en SS que luego convierte a color.
- Una segunda versión en la que el VS calcula el SS de la `SV_POSITION` y la envía al PS (con semántica `TEXCOORD1` por ejemplo).

Haz captura de cualquiera de las dos, pues deberían dar el mismo resultado (¡no olvides siempre dividir por w !).

6. Dibujando el screen space (2)

Aunque el shader anterior es el habitual (calcular el SS en el VS y pasarlo interpolado al PS) podemos plantearnos ser más eficientes y evitar la división por w en el PS.

Duplica el shader y ponle nombre `04ScreenSpaceNoDivW` de forma que en el VS se divida el SS por el `posCS.w` (que coincide con el `posSS.w`) y se evite así hacer la división en el PS.

¿Qué resultado esperas que salga? Si no sale igual, ajusta lo necesario antes de hacer la captura.

7. Z-buffer

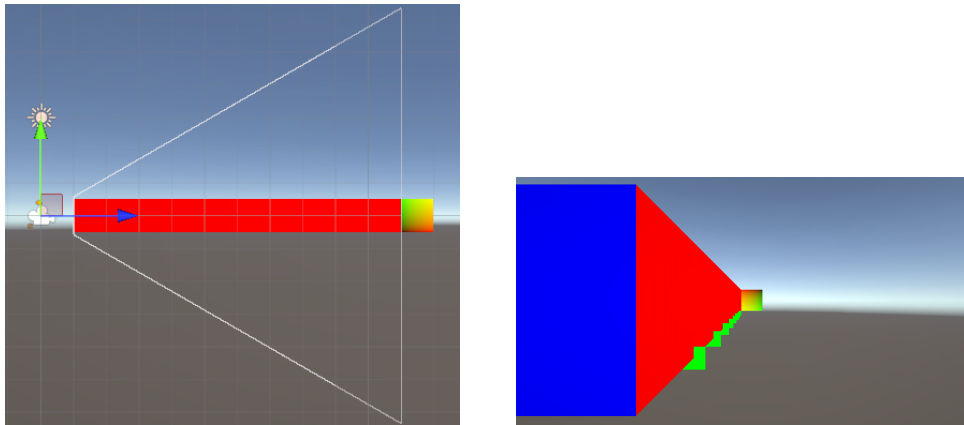
Para las siguientes prácticas vamos a cambiar de escena de forma que podamos ver mejor el efecto de la profundidad. Crea una escena nueva con:

- Cámara: L: (0, 0, 0), R: (0, 0, 0) . Configura el plano cercano en 1 y el lejano en 11 (un rango mucho menor que el que Unity utiliza por defecto).
- Cubo1: L: (-1, 0, 6), R:(0, 0, 0), S: (1, 1, 10) .
- Cubo2: L: (0, 0, 11.48), R:(0, 0, 0), S: (1, 1, 1) .

A modo de “marcadores” coloca también ocho planos L: (-0.5, -0.5, {2,3,4,...,10}), R: (-90, 0, 0), S: (0.02, 0.02, 0.02) . Al estar en las posiciones Z de la 2 a la 10 son una pista visual de la división del cubo en 10 partes de longitud 1.

Las siguientes capturas muestran una vista isométrica lateral de la escena y la visión desde la cámara si al cubo 1 y a los marcadores se les asigna el material que pinta las normales de la práctica anterior y al cubo 2 la que pinta las coordenadas de textura.

Puedes comprobar que si se mueve ligeramente el cubo 1 reduciendo su coordenada X desaparece su cara frontal por quedar demasiado cerca de la cámara, y si se aleja el cubo dos también, por quedar demasiado lejos.



Crea un shader llamado `05Profundidad` que devuelva como color en el PS la *profundidad*. Aunque no es la forma más adecuada de convertir a escala de grises, puedes simplemente poner en todas las componentes del color la z (recuerda dividirla por la w).

¿Cuál es el sentido del z-buffer en tu plataforma? ¿En cuál de los dos planos está situado el 0? Puedes comprobar que la cámara que utiliza Unity en la pestaña *Scene* no utiliza la misma configuración de plano cercano y lejano que la pestaña *Game*. Al tratarse de un rango más amplio, hay que acercarse mucho para ver un color que no sea muy parecido al del plano lejano.

8. Profundidad discretizada

Duplica el shader anterior y llámalo `06ProfundidadDiscreta`. Haz que en lugar de verse de forma continua, todo el rango de profundidad entre 0 y 0.1 se pinte con el color de la profundidad $z=0$, entre 0.1 y 0.2 con el de 0.1, etc.

Gracias a los marcadores, observa cómo el 10% del espacio de profundidad (la franja de color más oscuro) se utiliza para más de la mitad del espacio. Prueba a extender el plano lejano (y acerca el cercano) para comprobar cómo el descenso es aún más rápido.

9. Profundidad lineal

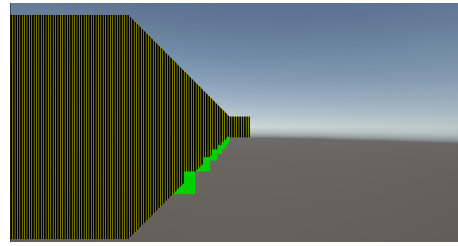
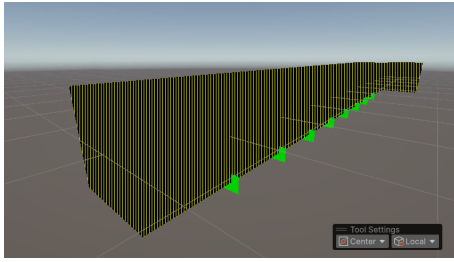
Es posible convertir la profundidad no lineal del z-buffer en un valor que sea lineal con la distancia a la cámara. Aunque podría hacerse el cálculo *a mano*, Unity dispone de varias funciones que lo hacen. Una de ellas es `Linear01DepthFromNear` que recibe el valor de z (sin dividir por w) y los parámetros del ZBuffer (variable de Unity `_ZBufferParams`) y convierte el valor del ZBuffer a espacio lineal (0 para el plano cercano).

Utilízalo en un nuevo shader `07ProfundidadLineal` que pinte ese valor también discretizándolo como antes. Dado que los marcadores dividen también el espacio de profundidad en 10 partes, deberían coincidir con los cambios de color.

10. Barras

Implementa un shader `08Barras` que pinte la geometría con barras verticales de color amarillo de un píxel de ancho y separadas por tres píxeles de color negro.

Para hacerlo utiliza la variable que Unity proporciona a los shaders `_ScreenParams` que tiene en sus campos x e y el tamaño del área de renderizado.



11. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Los ficheros de shaders y capturas indicados anteriormente con los nombres exactos.
- Incluir además un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.